

Pandas

Veri Madenciliği - Hafta 5

Giriş: Veri Bilimi ve Python Eko-Sistemi

Bu ders, veri madenciliği ve makine öğrenmesi algoritmaları bağlamında yapısal veri analizine odaklanacaktır.

Veri bilimi projelerinde kullanılan temel araçları tanıma.

Python'ın veri analizindeki merkezi rolü.

NumPy, SciPy, Matplotlib ve Pandas arasındaki ilişki.



Pandas Kütüphanesine Giriş

Pandas, veri analizi ve manipülasyonu için tasarlanmış açık kaynak kodlu bir Python kütüphanesidir.

'Panel Data' (Panel Verisi) kavramından adını alır.

Yüksek performanslı, kullanımı kolay veri yapıları sunar.

Özellikle etiketli (labeled) verilerle çalışmak için idealdir.



Pandas'ın Tarihsel Gelişimi ve Amacı

Pandas, AQR Capital Management'ta Wes McKinney tarafından geliştirilmiştir. Başlangıçta finansal veri analizi ihtiyaçlarını karşılamak üzere tasarlanmıştır. R'daki 'data.frame' yapısına benzer, ancak daha optimize edilmiş araçlar sunar. Zaman serisi verileri için benzersiz yeteneklere sahiptir.

Pandas'ın Temel Yapı Taşları

Pandas temel olarak iki ana veri yapısı kullanır: Series ve DataFrame. Bu yapılar, veri manipülasyonu ve temizliği için gerekli esnekliği sağlar. Yapısal veri analizi, bu iki veri yapısı etrafında şekillenir.

Veri Yapısı 1: Series (Seriler)

Seriler, tek boyutlu etiketli (labeled) indekslenmiş verilerdir. NumPy dizilerinden farkı, etiketli bir indekse sahip olmalarıdır. Bir sütun veya bir satır olarak düşünülebilir. Seriler heterojen veri tiplerini tutabilir (örneğin, tam sayı, float, string).

Series Oluşturma Yöntemleri

Bir Seriyi oluşturmanın temel yolu 'pd.Series()' fonksiyonunu kullanmaktır. Python listeleri, NumPy dizileri veya sözlükler (dictionaries) kullanılarak oluşturulabilirler. Sözlükten oluşturulduğunda, sözlüğün anahtarları otomatik olarak indeks (etiket) haline gelir.



Uygulama: Python Listesinden Series Oluşturma

```
import pandas as pd  
veri = [10, 20, 30, 40]  
s1 = pd.Series(veri)
```

Çıktı, otomatik olarak 0'dan başlayan varsayılan tamsayı indeksi gösterir.
dtype: Veri tipini belirtir (örneğin int64).



Series İndeksi ve Etiketleme

Series'in en güçlü yönü, varsayılan sayısal indeks yerine özel etiketler (string veya başka tipler) kullanabilme yeteneğidir.
'index' parametresi kullanılarak etiketler belirlenir.
Etiketler, veri erişimini ve okunabilirliğini artırır.



Series Özellikleri (Attributes)

Series nesnelerinin temel özelliklerinin incelenmesi:

'`values`': Series içerisindeki veriyi bir NumPy dizisi olarak döndürür.

'`index`': İndeks etiketlerini döndürür.

'`dtype`': Veri tipini döndürür.

'`shape`': Series'in boyutunu (tek elemanlı tuple) döndürür.



Series Üzerinde Temel İşlemler

Series, NumPy'dan miras aldığı vektörel işlemleri destekler. Toplama, çıkarma, çarpma ve bölme gibi işlemler tüm elemanlara uygulanır (broadcasting). Bu işlemler, döngü yazma zorunluluğunu ortadan kaldırır ve performansı artırır.



Series'te Eksik Veri Yönetimi

Series, eksik değerleri (Missing Values) temsil etmek için 'NaN' (Not a Number) kullanır.

'isnull()' veya 'isna()' fonksiyonları, hangi elemanların eksik olduğunu kontrol eder.

Eksik veriyi doldurmak ('.fillna()') veya kaldırmak ('.dropna()') temel veri temizleme işlemlerindendir.

Veri Yapısı 2: DataFrame (Veri Çerçevesi)

DataFrame, Pandas'ın en temel ve en çok kullanılan iki boyutlu veri yapısıdır. Sütunlar ve satırlardan oluşan tablolar olarak düşünülebilir. İlişkisel veritabanı tablolarına veya Excel elektronik tablolarına benzer. Her sütun bir Series nesnesidir ve tüm sütunlar aynı indeksi paylaşır.

DataFrame'in Yapısal Bileşenleri

DataFrame üç ana bileşenden oluşur:

1. Satır İndeksi (Row Index): Satır etiketleri.
2. Sütunlar (Columns): Sütun başlıkları, her biri bir Series'i temsil eder.
3. Veri Matrisi (Data Matrix): Gerçek veri değerleri.

DataFrame Oluşturma Temelleri

En yaygın DataFrame oluşturma yöntemlerinden biri, Python sözlüklerini kullanmaktır.

Sözlüğün anahtarları sütun isimleri olur.

Sözlüğün değerleri (listeler), o sütunun verilerini oluşturur.

Uygulama: Sözlükten DataFrame Oluşturma

```
import pandas as pd
data_dict = {'Ad': ['Ali', 'Ayşe', 'Mehmet'], 'Yaş': [25, 30, 28]}
df = pd.DataFrame(data_dict)
```

Bu yapı, verinin okunabilirliğini ve yapısal bütünlüğünü garanti eder.

DataFrame Oluşturma: Listeler ve NumPy Dizileri

Listelerin listesi (nested lists) veya NumPy iki boyutlu dizileri ('ndarray') kullanılarak da DataFrame oluşturulabilir.

Bu durumda, 'columns' parametresi belirtilmezse, sütun isimleri varsayılan olarak 0, 1, 2... olur.

Bu yöntem, özellikle makine öğrenmesi algoritmalarından gelen çıktıları tabloya dönüştürmede faydalıdır.



DataFrame Meta Verilerinin İncelenmesi 1

Veri çerçevelerini incelemek için kullanılan temel fonksiyonlardan biri 'df.info()' komutudur.

Bu komut, DataFrame hakkında hızlı bir genel bakış sağlar:

- Satır sayısı (Index aralığı)
- Sütun sayısı ve isimleri
- Her sütunun veri tipi ('dtype')
- Eksik olmayan (Non-Null) değer sayısı.

DataFrame Meta Verilerinin İncelenmesi 2

'df.describe()' komutu, sayısal sütunların temel istatistiksel özetini sunar. Standart sapma, ortalama, minimum, maksimum ve çeyreklik (quartile) değerlerini içerir. Verinin dağılımını anlamak, aykırı değerleri (outliers) tespit etmek için kritik öneme sahiptir.



Veri Önizleme: head() ve tail()

'df.head(n)' komutu, veri çerçevesinin ilk 'n' satırını gösterir (varsayılan: 5).
'df.tail(n)' komutu, veri çerçevesinin son 'n' satırını gösterir (varsayılan: 5).
Bu fonksiyonlar, veriyi yükledikten veya manipüle ettikten sonra hızlı bir kontrol yapmak için kullanılır.

Veri Seçiminin Önemi

Veri analizi sürecinde, verinin belirli kısımlarına erişmek ve bunları izole etmek esastır.

Pandas, verilerin seçim işleminde son derece esnek birçok yöntem sunar. Seçim, satırlara, sütunlara veya hem satırlara hem de sütunlara aynı anda uygulanabilir.

Temel seçim yöntemleri: Etikete dayalı ('loc') ve konuma dayalı ('iloc').

Konuma Dayalı Seçim: iloc Komutu

'iloc' komutu, veriyi tamsayı konumu (position) baz alarak seçer. Seçim, Python'daki listeleme (slicing) mantığına çok benzerdir (0'dan başlar). 'df.iloc[satır_konumu, sütun_konumu]' şeklinde kullanılır. Tıpkı Python slicing'de olduğu gibi, bitiş konumu dahil değildir (exclusive).

iloc Detaylı Kullanım Senaryoları

Tek satır seçimi: `'df.iloc[5]'` (6. satır)

Satır aralığı seçimi: `'df.iloc[0:10]'` (ilk 10 satır)

Belirli satır ve sütun seçimi: `'df.iloc[[0, 2, 4], [0, 1]]'`

Tüm sütunları seçme: `'df.iloc[:, 1:4]'`

Etikete Dayalı Seçim: loc Komutu

'loc' komutu, veriyi etiket (label) baz alarak seçer.

Hem satır indeksi hem de sütun isimleri kullanılır.

'df.loc[satır_etiketi, sütun_etiketi]' şeklinde kullanılır.

****Önemli Fark:**** 'loc' kullanıldığında, aralık seçimlerinde bitiş etiketi DAHİLDİR (inclusive).

loc Detaylı Kullanım Senaryoları

Tek bir sütunu etiketle seçme: `'df.loc[:, 'Ad']'`

Etiket aralığı ile satır seçme: `'df.loc['A':'G']'` (A'dan G etiketine kadar, G dahil)

Belirli satır ve sütun etiketleriyle seçim: `'df.loc[['Ayşe', 'Ali'], ['Yaş', 'Şehir']]'`

Boolean (Mantıksal) indeksleme

DataFrame'deki değerleri bir koşula göre filtrelemek için kullanılır. Koşul, Series objesi boyutunda 'True'/'False' değerlerinden oluşan bir Boolean Serisi üretir.

Sadece 'True' olduğu yerlerdeki satırlar DataFrame'e dahil edilir. Örnek: `df[df['Yaş'] > 25]` (Yaşı 25'ten büyük olanları seç).

Çoklu Koşullu Seçim

Birden fazla koşulu birleştirmek için mantıksal operatörler kullanılır:

AND için '&' (ampersand) operatörü.

OR için '|' (pipe) operatörü.

Koşulların parantez içine alınması zorunludur: `'df[(df['Yaş'] > 20) & (df['Şehir'] == 'Ankara')]`

Bu, karmaşık filtreleme işlemlerinin temelidir.

Veri Çerçevesi Manipülasyonu 1: Yeni Sütun Ekleme

Yeni sütun eklemek, var olan sütunlar üzerinde hesaplamalar yapmanın en kolay yoludur.

Basit atama yöntemi kullanılır: `df['Yeni Sütun Adı'] = hesaplama veya değer'`
Vektörel işlemler sayesinde, tüm satırlar için hızlıca değer ataması yapılabilir (Örnek: `Ücret * 1.18`).

Veri Çerçevesi Manipülasyonu 2: Sütun Silme

'df.drop()' metodu kullanılarak sütunlar (veya satırlar) silinebilir. Sütun silmek için 'axis=1' (veya 'axis='columns') parametresi belirtilmelidir. Kalıcı silme işlemi için 'inplace=True' parametresi kullanılır. Alternatif olarak, 'del df['Sütun Adı']' yapısı da kullanılabilir.

Dış Kaynaklardan Veri Okuma: read_csv

'pd.read_csv()' fonksiyonu, en yaygın kullanılan veri okuma yöntemidir. Dosya yolunu veya URL'yi girdi olarak alır. Yüzlerce parametreye sahiptir; bunlar arasında ayırıcı ('sep'), başlık satırı ('header') ve eksik değer gösterimi ('na_values') ayarları bulunur.

read_csv Parametreleri: Delimiter ve Encoding

Ayırıcı ('sep'): Veri dosyasındaki sütunları ayıran karakteri belirtir (örneğin: ';' veya ' ')

Kodlama ('encoding'): Dosyanın hangi karakter setinde yazıldığını belirtir (örneğin: 'utf-8', 'latin-1').

Yanlış encoding seçimi, özellikle Türkçe karakterlerde veri kaybına neden olabilir.

read_csv Parametreleri: Index ve Header

'index_col': Hangi sütunun DataFrame'in indeksi olarak kullanılacağını belirtir.
'header': Başlık satırının hangi satırda olduğunu belirtir (varsayılan: 0). Başlık yoksa 'header=None' kullanılır.



Veri Çerçevesini Dışa Aktarma: to_csv

İşlenen DataFrame'i diske kaydetmek için 'df.to_csv()' fonksiyonu kullanılır. Kaydedilen dosya adı ve yolu zorunlu parametrelerdir. 'index=False' parametresi, DataFrame indeksinin çıktı dosyasına yazılmasını engeller (genellikle tercih edilen ayar).

Diğer I/O Fonksiyonları

Pandas, CSV dışında birçok formatı destekler:

'read_excel()' / 'to_excel()': Excel dosyaları için.

'read_json()' / 'to_json()': JSON formatındaki veriler için.

'read_sql()': Veritabanlarından veri çekmek için (SQLAlchemy gerektirir).

Veri Birleştirme Kavramı

Gerçek dünya analizlerinde veriler genellikle parçalar halinde gelir.

Pandas, iki temel birleştirme operasyonu sunar:

1. Ekleme (Stacking) / Birleştirme: 'concat'
2. İlişkilendirme (Joining) / Birleştirme: 'merge' ve 'join'

Dikey ve Yatay Birleştirme: Concat

'pd.concat()' fonksiyonu, veri çerçevelerini dikey ('axis=0') veya yatay ('axis=1') olarak birleştirmek için kullanılır.

Dikey birleştirme (append), aynı sütunlara sahip veri çerçevelerini alt alta ekler.

Yatay birleştirme, aynı indekse sahip veri çerçevelerini yan yana ekler.

İlişkisel Birleştirme: Merge (SQL Join Benzeri)

'pd.merge()' fonksiyonu, SQL JOIN işlemlerine eşdeğerdir ve iki DataFrame'i ortak sütunlar (anahtarlar) üzerinden birleştirir. Genellikle en esnek birleştirme yöntemidir. Temel parametreler: Birleştirilecek sütun ('on'), birleştirme tipi ('how').

Merge Tipleri (How Parametresi)

'how='inner': Yalnızca her iki veri çerçevesinde de ortak olan anahtarları tutar (Kesişim).

'how='outer': Tüm anahtarları tutar, eşleşmeyen yerlere NaN atar (Birleşim).

'how='left': Sol DataFrame'deki tüm anahtarları tutar.

'how='right': Sağ DataFrame'deki tüm anahtarları tutar.



İndeks Üzerinden Birleştirme: Join

'df1.join(df2)' metodu, iki DataFrame'i indeksleri üzerinden birleştirir.

'merge"'e göre daha sade bir söz dizimine sahiptir.

'how' parametresi (inner, outer, left, right) 'merge' ile aynı mantıkta çalışır.

Genellikle, bir DataFrame'e indeksine göre bir Series (veya tek sütunlu DF) eklemek için kullanılır.



Veri Temizliği: Eksik Değer İşlemleri

Eksik veriyi ortadan kaldırmak için '`df.dropna()`' kullanılır. Sadece NaN içeren satırları veya sütunları siler.

Eksik veriyi doldurmak için '`df.fillna(değer)`' kullanılır.

Doldurma stratejileri: Ortalama ile doldurma ('`df.mean()`'), medyan ile doldurma veya komşu değerlerle doldurma ('`method='ffill'` veya '`method='bfill'`').

Veri Dönüşümü: apply() ve map()

'df.apply()': Satır veya sütun bazında karmaşık fonksiyonları uygulamak için kullanılır.

'df['Sütun'].map()': Bir Series üzerindeki her elemanı bir sözlüğe göre dönüştürmek veya bir fonksiyondan geçirmek için kullanılır.

Bu metotlar, standart Pandas işlemlerinin yetersiz kaldığı durumlarda esneklik sağlar.



Gruplama ve Toplama: groupby()

Veri çerçevesini bir veya daha fazla sütunun değerlerine göre gruplara ayırır. 'Böl-Uygula-Birleştir' (Split-Apply-Combine) paradigmasını uygular. Kullanımı: 'df.groupby('grup_sütunu').fonksiyon()' (Örn: '.mean()', '.sum()', '.count()'). Bu işlem, kategorik değişkenlere göre istatistiksel özetler elde etmeyi sağlar.

Pivot Tabloları ve Çapraz Tablolar

'pd.pivot_table()': Veritabanı veya elektronik tablo pivot işlevine benzer. Üç anahtar parametre: 'index' (satır), 'columns' (sütun), 'values' (hesaplama yapılacak değer).

'pd.crosstab()': İki veya daha fazla kategorik değişken arasındaki frekansları saymak için kullanılır.

Kategorik Veri Yönetimi

Metin tabanlı, tekrar eden değerler içeren sütunlar için 'dtype='category' kullanılabilir.

Kategorik veri tipi, veriyi sayısal kodlarla temsil ederek bellek kullanımını önemli ölçüde azaltır.

Kategorik verilerle karşılaştırma ve sıralama işlemleri daha hızlıdır.



Zaman Serisi Verileriyle Çalışma

Pandas, zaman serisi verileri için optimize edilmiştir.
'DatetimeIndex', veriye zaman etiketlerine göre erişim sağlar.
'resample()' metodu: Veriyi daha düşük (aylık, yıllık) veya daha yüksek (saniyelik) frekanslara dönüştürmek için kullanılır.

Performans İpuçları ve En İyi Uygulamalar

Python döngülerinden kaçınin, bunun yerine vektörel Pandas/NumPy fonksiyonlarını kullanın.

Büyük veri setlerinde 'apply()' yerine 'itertuples()' veya 'iterrows()' kullanın (ancak bu, genellikle yine de döngüden kaçınma ilkesine aykırıdır).

Mümkün olduğunda 'dtype'ları (özellikle 'category') doğru ayarlayın.

Bellek sınırlarını aşmamak için 'chunksize' parametresiyle büyük dosyaları parça parça okuyun.



Pandas ve NumPy İlişkisi

Pandas, temel olarak NumPy üzerine inşa edilmiştir.

Tüm DataFrame ve Series değerleri, aslında temel düzeyde NumPy dizileri ('.values') olarak saklanır.

Bu, NumPy'in yüksek performanslı matematiksel ve vektörel işlemlerini Pandas'a taşır.

Karmaşık sayısal hesaplamalar için NumPy'i kullanmak, performansı artırır.

Veri Analizinde Pandas'ın Rolü

Pandas, makine öğrenmesi algoritmalarına veri beslemeden önceki en kritik adımı, yani Veri Ön İşleme (Preprocessing) adımını yönetir. Bu adımlar: Veri temizleme, eksik değer yönetimi, aykırı değer tespiti ve özellik mühendisliği (Feature Engineering). Sağlam ve temiz veriler olmadan makine öğrenmesi modelleri doğru sonuçlar üretemez.

Sonuç: Pandas'ın Yetkinlikleri

Pandas, yapısal veri analizi için vazgeçilmez bir araçtır.

Öğrenciler, Series ve DataFrame oluşturmayı, seçim işlemlerini (loc, iloc) ve I/O işlemlerini (read_csv, to_csv) öğrendiler.

Bu temel bilgiler, daha karmaşık veri madenciliği ve makine öğrenmesi görevleri için sağlam bir zemin hazırlar.

Veri bilimcileri için Pandas, günlük iş akışının %80'ini oluşturan temizlik ve hazırlık aşamalarını basitleştirir.

Pandas

Süleyman **EZDEMİR**

suleyman.ezdemir@giresun.edu.tr

